

Digital Design Diploma

Mini Project : Spartan6 DSP48A1

Name:	Roaa Atef
--------------	------------------

Questasem Part:

1. Verilog code:

```
1 module reg_plus_mux (clk,rst ,reg_en,D,D_MUX);
2 parameter PIPELINE =0;
3 parameter WIDTH=1;
4 parameter RST_TYPE="SYNC";
5 //////////////////////////////////////////////////
6 input clk,rst;
7 input [WIDTH-1:0]D;
8 input reg_en;
9 output [WIDTH-1:0]D_MUX;
10 reg [WIDTH-1:0] D_FF;
11 //////////////////////////////////////////////////
12 assign D_MUX =(PIPELINE==0)?D:D_FF;
13 generate
14 if (RST_TYPE=="SYNC") begin
15 always @(posedge clk) begin
16 if(rst)
17 D_FF<=0;
18 else if((reg_en))
19 begin
20 D_FF<=D;
21 end
22 end
23 end
24 else begin
25 always @(posedge clk or posedge rst ) begin
26 if(rst)
27 D_FF<=0;
28 else if((reg_en))
29 begin
30 D_FF<=D;
31 end
32 end
33 end
34 endgenerate
35 endmodule
36
```

```
1
2 module DSP (
3 input [17:0] A,
4 input [17:0] B,
5 input [47:0] C,
6 input [17:0] D,
7 input CARRYIN,
8 output [35:0] M,
9 output [47:0] P,
10 output CARRYOUT,
11 output CARRYOUTF,
12 input CLK,
13 input [7:0]OPMODE,
14 input CEA,CEB,CEC,CECARRYIN,CED,CEM,CEOPMODE,CEP,
15 input RSTA,RSTB,RSTC,RSTCARRYIN,RSTD,RSTM,RSTOPMODE,RSTP,
16 input [47:0] PCIN,
17 output [47:0] PCOUT,
18 output [17:0] BCOUT,
19 input [17:0] BCIN
20 );
21 //////////////////////////////////////////////////
22 parameter A0REG=0;
23 parameter A1REG=1;
24 parameter B0REG=0;
25 parameter B1REG=1;
26 parameter CREG=1;
27 parameter DREG=1;
28 parameter MREG=1;
29 parameter PREG=1;
30 parameter CARRYINREG=1;
31 parameter CARRYOUTREG=1;
32 parameter OPMODEREG=1;
33 parameter CARRYINSEL="OPMODES";
34 parameter B_INPUT="DIRECT";
35 parameter RSTTYPE="SYNC";
36 //////////////////////////////////////////////////
37 //////////////////////////////////////////////////
38 wire [17:0] A_reg;
39 wire [17:0] B_reg;
40 wire [47:0] C_reg;
41 wire [17:0] D_reg;
42 wire [17:0] A_reg2;
43 wire [17:0] B_reg2;
44 reg cout;
45 wire CARRYOUT_reg;
46 wire CARRYIN_reg;
47 reg CARRYIN_mux;
48 reg [17:0] B_mux;
49 wire [7:0] OPMODE_reg;
50 wire [35:0] M_reg;
51 wire [47:0] P_reg;
52 //////////////////////////////////////////////////
53 reg [17:0] B_D_ADD_SUB;
54 reg [17:0] op_mode_4_mux;
55 reg [35:0] MUL;
```

```

56 reg [47:0] MUX_x;
57 reg [47:0] MUX_z;
58 reg [47:0] z_x_ADDER;
59 ///////////////////////////////////////////////////
60 assign BCOUT= B_reg2;
61 assign M=M_reg;
62 assign P=P_reg;
63 assign PCOUT=P_reg;
64 assign CARRYOUT=CARRYOUT_reg;
65 assign CARRYOUTF=CARRYOUT_reg;
66 ///////////////////////////////////////////////////
67 generate
68 if (A0REG==0) begin
69     reg_plus_mux    #(.PIPELINE(A0REG),.WIDTH(18),.RST_TYPE(RSTTYPE)) reg_a (CLK,RSTA ,CEA,A,A_reg);
70
71 end
72 else begin
73     reg_plus_mux    #(.PIPELINE(A0REG),.WIDTH(18),.RST_TYPE(RSTTYPE)) reg_a (CLK,RSTA ,CEA,A,A_reg);
74 end
75
76 ///////////////////////////////////////////////////
77
78 if (B0REG==0) begin
79     reg_plus_mux    #(.PIPELINE(B0REG),.WIDTH(18),.RST_TYPE(RSTTYPE)) reg_b (CLK,RSTB ,CEB,B_mux,B_reg);
80
81 end
82 else begin
83     reg_plus_mux    #(.PIPELINE(B0REG),.WIDTH(18),.RST_TYPE(RSTTYPE)) reg_b (CLK,RSTB ,CEB,B_mux,B_reg);
84 end
85
86 ///////////////////////////////////////////////////
87
88 if (CREG==0) begin
89     reg_plus_mux    #(.PIPELINE(CREG),.WIDTH(48),.RST_TYPE(RSTTYPE)) reg_c (CLK,RSTC ,CEC,C,C_reg);
90
91 end
92 else begin
93     reg_plus_mux    #(.PIPELINE(CREG),.WIDTH(48),.RST_TYPE(RSTTYPE)) reg_c (CLK,RSTC ,CEC,C,C_reg);
94 end
95
96 ///////////////////////////////////////////////////
97
98 if (DREG==0) begin
99     reg_plus_mux    #(.PIPELINE(DREG),.WIDTH(18),.RST_TYPE(RSTTYPE)) reg_d (CLK,RSTD ,CED,D,D_reg);
100
101 end
102 else begin
103     reg_plus_mux    #(.PIPELINE(DREG),.WIDTH(18),.RST_TYPE(RSTTYPE)) reg_d (CLK,RSTD ,CED,D,D_reg);
104 end
105
106 ///////////////////////////////////////////////////
107
108 if (A1REG==0) begin
109     reg_plus_mux    #(.PIPELINE(A1REG),.WIDTH(18),.RST_TYPE(RSTTYPE)) reg_a1 (CLK,RSTA ,CEA,A_reg,A_reg2);
110

```

```

111 end
112 else begin
113     reg_plus_mux    #(.PIPELINE(A1REG),.WIDTH(18),.RST_TYPE(RSTTYPE)) reg_a1 (CLK,RSTA ,CEA,A_reg,A_reg2);
114 end
115
116 ///////////////////////////////////////////////////
117
118 if (B1REG==0) begin
119     reg_plus_mux    #(.PIPELINE(B1REG),.WIDTH(18),.RST_TYPE(RSTTYPE)) reg_b1 (CLK,RSTB ,CEB,op_mode_4_mux,B_reg2);
120 end
121
122 else begin
123     reg_plus_mux    #(.PIPELINE(B1REG),.WIDTH(18),.RST_TYPE(RSTTYPE)) reg_b1 (CLK,RSTB ,CEB,op_mode_4_mux,B_reg2);
124 end
125
126 ///////////////////////////////////////////////////
127
128 if (MREG==0) begin
129     reg_plus_mux    #(.PIPELINE(MREG),.WIDTH(36),.RST_TYPE(RSTTYPE)) reg_m (CLK,RSTM ,CEM,MUL,M_reg);
130 end
131
132 else begin
133     reg_plus_mux    #(.PIPELINE(MREG),.WIDTH(36),.RST_TYPE(RSTTYPE)) reg_m (CLK,RSTM ,CEM,MUL,M_reg);
134 end
135
136 ///////////////////////////////////////////////////
137
138 if (PREG==0) begin
139     reg_plus_mux    #(.PIPELINE(PREG),.WIDTH(48),.RST_TYPE(RSTTYPE)) reg_p (CLK,RSTP,CEP,z_x_ADDER,P_reg);
140 end
141
142 else begin
143     reg_plus_mux    #(.PIPELINE(PREG),.WIDTH(48),.RST_TYPE(RSTTYPE)) reg_p (CLK,RSTP ,CEP,z_x_ADDER,P_reg);
144 end
145
146 ///////////////////////////////////////////////////
147
148 if (CARRYINREG==0) begin
149     reg_plus_mux    #(.PIPELINE(CARRYINREG),.WIDTH(1),.RST_TYPE(RSTTYPE)) reg_carr (CLK,RSTCARRYIN ,CECARRYIN,CARRYIN_mux,CARRYIN_reg);
150 end
151
152 else begin
153     reg_plus_mux    #(.PIPELINE(CARRYINREG),.WIDTH(1),.RST_TYPE(RSTTYPE)) reg_carr (CLK,RSTCARRYIN ,CECARRYIN,CARRYIN_mux,CARRYIN_reg);
154 end
155
156 ///////////////////////////////////////////////////
157
158 if (CARRYOUTREG==0) begin
159     reg_plus_mux    #(.PIPELINE(CARRYOUTREG),.WIDTH(1),.RST_TYPE(RSTTYPE)) reg_carro (CLK,RSTCARRYIN,CECARRYIN,cout,CARRYOUT_reg);
160 end
161
162 else begin
163     reg_plus_mux    #(.PIPELINE(CARRYOUTREG),.WIDTH(1),.RST_TYPE(RSTTYPE)) reg_carro (CLK,RSTCARRYIN ,CECARRYIN,cout,CARRYOUT_reg);
164 end
165

```

```

166 ///////////////////////////////////////////////////
167
168 if (OPMODEREG==0) begin
169     reg_plus_mux    #(.PIPELINE(OPMODEREG),.WIDTH(8),.RST_TYPE(RSTTYPE)) reg_op (CLK,RSTOPMODE ,CEOPMODE,OPMODE,OPMODE_reg);
170 end
171
172 else begin
173     reg_plus_mux    #(.PIPELINE(OPMODEREG),.WIDTH(8),.RST_TYPE(RSTTYPE)) reg_op (CLK,RSTOPMODE ,CEOPMODE,OPMODE,OPMODE_reg);
174 end
175
176 endgenerate
177 ///////////////////////////////////////////////////
178 always @(*) begin
179     if(B_INPUT=="DIRECT") begin
180         B_mux=B;
181     end
182     else if (B_INPUT=="CASCADE")begin
183         B_mux=BCIN;
184     end
185     else begin
186         B_mux=0;
187     end
188 ///////////////////////////////////////////////////
189 if(OPMODE_reg[6]==0) begin
190     B_D_ADD_SUB=B_reg+D_reg;
191 end
192 else begin
193     B_D_ADD_SUB=D_reg-B_reg;
194 end
195 ///////////////////////////////////////////////////
196 if(OPMODE_reg[4]==0) begin
197     op_mode_4_mux=B_reg;
198 end
199 else begin
200     op_mode_4_mux=B_D_ADD_SUB;
201 end
202 ///////////////////////////////////////////////////
203 MUL= B_reg2A_reg2;
204 ///////////////////////////////////////////////////
205 case (OPMODE_reg[1:0])
206 2'b00: MUX_x=0;
207 2'b01: MUX_x={12'b0,M_reg};
208 2'b10: MUX_x=P_reg;
209 2'b11: MUX_x={D_reg[11:0],A_reg2[17:0],B_reg2[17:0]};
210 endcase
211 ///////////////////////////////////////////////////
212 if(CARRYINSEL=="OPMODE5") begin
213     CARRYIN_mux=OPMODE_reg[5];
214 end
215 else if (CARRYINSEL=="CARRYIN")begin
216     CARRYIN_mux=CARRYIN;
217 end
218 else begin
219     CARRYIN_mux=0;
220 end
221 ///////////////////////////////////////////////////

```

```

220 //////////////////////////////////////////////////
221 case (OPMODE_reg[3:2])
222 2'b00: MUX_z=0;
223 2'b01: MUX_z=PCIN;
224 2'b10: MUX_z=P_reg;
225 2'b11: MUX_z=C_reg;
226 endcase
227 //////////////////////////////////////////////////
228 if(OPMODE_reg[7]==1) begin
229 {cout,z_x_ADDER}=MUX_z-(MUX_x+CARRYIN_reg);
230 end
231 else begin
232 {cout,z_x_ADDER}=MUX_z+MUX_x+CARRYIN_reg;
233 end
234 end
235 endmodule

```

2. Tb:

```

1 module DSP_tb();
2 reg [17:0] A;
3 reg [17:0] B;
4 reg [47:0] C;
5 reg [17:0] D;
6 reg CARRYIN;
7 wire [35:0] M;
8 wire [47:0] P;
9 wire CARRYOUT;
10 wire CARRYOUTF;
11 reg CLK;
12 reg [7:0] OPMODE;
13 reg CEA,CEB,CEC,CECARRYIN,CED,CEM,CEOPMODE,CEP;
14 reg RSTA,RSTB,RSTC,RSTCARRYIN,RSTD,RSTM,RSTOPMODE,RSTP;
15 reg [47:0] PCIN;
16 wire [47:0] PCOUT;
17 wire [17:0] BCOUT;
18 reg [17:0] BCIN;
19 //////////////////////////////////////////////////
20 parameter A0REG=0;
21 parameter A1REG=1;
22 parameter B0REG=0;
23 parameter B1REG=1;
24 parameter CREG=1;
25 parameter DREG=1;
26 parameter MREG=1;
27 parameter PREG=1;
28 parameter CARRYINREG=1;
29 parameter CARRYOUTREG=1;
30 parameter OPMODEREG=1;
31 parameter CARRYINSEL="OPMODE5";
32 parameter B_INPUT="DIRECT";
33 parameter RSTTYPE="SYNC";
34 //////////////////////////////////////////////////
35 reg [17:0] W1;
36 reg [47:0] W2,W3;
37 reg [48:0] W4;
38 DSP dsp (
39 A,
40 B,
41 C,
42 D,
43 CARRYIN,
44 M,
45 P,
46 CARRYOUT,
47 CARRYOUTF,
48 CLK,
49 OPMODE,
50 CEA,CEB,CEC,CECARRYIN,CED,CEM,CEOPMODE,CEP,
51 RSTA,RSTB,RSTC,RSTCARRYIN,RSTD,RSTM,RSTOPMODE,RSTP,
52 PCIN,
53 PCOUT,
54 BCOUT,
55 BCIN

```

```

56     );
57     //////////////////////////////////
58     initial begin
59         CLK=0;
60         forever
61             #1 CLK=~CLK;
62     end
63     //////////////////////////////////
64     integer i;
65     initial begin
66         //rst test
67         RSTA=1; RSTB=1; RSTC=1; RSTD=1;
68         RSTOPMODE=1; RSTCARRYIN=1;
69         RSTP=1; RSTM=1;
70         //////////////////////////////////
71         A=0; B=0; C=0; D=0;
72         BCIN=0; PCIN=0;
73         CARRYIN=0; OPMODE=0;
74         CEA=0; CEB=0; CEC=0; CECARRYIN=0; CED=0; CEM=0; CEOPMODE=0; CEP=0;
75         repeat (8) begin
76             @(negedge CLK);
77         end
78         //////////////////////////////////
79         RSTA=0; RSTB=0; RSTC=0; RSTD=0;
80         RSTOPMODE=0; RSTCARRYIN=0;
81         RSTP=0; RSTM=0;
82         CEA=1; CEB=1; CEC=1; CECARRYIN=1; CED=1; CEM=1; CEOPMODE=1; CEP=1;
83         // test first adder (add), second adder (add) // 12
84         A=18'd1; B=18'd1 ; C= 48'd1; D=18'd1;
85         PCIN=48'd10;
86         OPMODE[6]=1'b0;
87         OPMODE[4]=1'b1;
88         OPMODE[1:0] =2'b01;
89         OPMODE[3:2]=2'b01;
90         OPMODE[7]=1'b0;
91         OPMODE[5]=1'b0;
92         W1=B+D;
93         W2=W1*A;
94         W3=W2+PCIN;
95         repeat(4) begin
96             @(negedge CLK);
97         end
98         if (P!=W3)
99             begin
100                 $display ("error");
101                 $display("P=%d,W3=%d",P,W3);
102                 $stop;
103             end
104             else begin
105                 $display("success");
106             end
107             //////////////////////////////////
108             //test first adder (sub) , second adder (add) //14
109             A=18'd1; B=18'd1 ; C= 48'd1; D=18'd5;
110             PCIN=48'd10;

```

```

111 OPMODE[6]=1'b1;
112 OPMODE[4]=1'b1;
113 OPMODE[1:0] =2'b01;
114 OPMODE[3:2]=2'b01;
115 OPMODE[7]=1'b0;
116 OPMODE[5]=1'b0;
117 W1=D-B;
118 W2=W1*A;
119 W3=W2+PCIN;
120 repeat(4) begin
121     @(negedge CLK);
122 end
123 if (P!=W3)
124     begin
125         $display("error");
126         $display("P=%d,W3=%d",P,W3);
127         $stop;
128     end
129     else begin
130 $display("success");
131 end
132 //////////////////////////////////////////////////
133 //test second adder (sub) , first adder (add) //8
134 A=18'd1; B=18'd1 ; C= 48'd1; D=18'd1;
135 PCIN=48'd10;
136 OPMODE[6]=1'b0;
137 OPMODE[4]=1'b1;
138 OPMODE[1:0] =2'b01;
139 OPMODE[3:2]=2'b01;
140 OPMODE[7]=1'b1;
141 OPMODE[5]=1'b0;
142 W1=D+B;
143 W2=W1*A;
144 W3=PCIN-W2;
145 repeat(4) begin
146     @(negedge CLK);
147 end
148 if (P!=W3)
149     begin
150         $display("error");
151         $display("P=%d,W3=%d",P,W3);
152         $stop;
153     end
154     else begin
155 $display("success");
156 end
157 //////////////////////////////////////////////////
158 //test second adder (sub) , first adder (sub) //1
159 A=18'd1; B=18'd1 ; C= 48'd1; D=18'd10;
160 PCIN=48'd10;
161 OPMODE[6]=1'b1;
162 OPMODE[4]=1'b1;
163 OPMODE[1:0] =2'b01;
164 OPMODE[3:2]=2'b01;
165 OPMODE[7]=1'b1;

```

```

166 OPMODE[5]=1'b0;
167 W1=D-B;
168 W2=W1*A;
169 W3=PCIN-W2;
170 repeat(4) begin
171     @(negedge CLK);
172 end
173 if (P!=W3)
174     begin
175         $display ("error");
176         $display("P=%d,W3=%d",P,W3);
177         $stop;
178     end
179     else begin
180 $display("success");
181 end
182 ///////////////////////////////////////////////////
183 //test second adder (add) , first adder (add),opmode_4_mux=B // ((B*A)+PCIN)=11
184 A=18'd1; B=18'd1 ; C= 48'd1; D=18'd10;
185 PCIN=48'd10;
186 OPMODE[6]=1'b0;
187 OPMODE[4]=1'b0;
188 OPMODE[1:0] =2'b01;
189 OPMODE[3:2]=2'b01;
190 OPMODE[7]=1'b0;
191 OPMODE[5]=1'b0;
192 W2=B*A;
193 W3=W2+PCIN;
194 repeat(4) begin
195     @(negedge CLK);
196 end
197 if (P!=W3)
198     begin
199         $display ("error");
200         $display("P=%d,W3=%d",P,W3);
201         $stop;
202     end
203     else begin
204 $display("success");
205 end
206 ///////////////////////////////////////////////////
207 //test second adder (add) , first adder (add) , cin // ((D+B)*A)+PCIN+CIN=22;
208 A=18'd1; B=18'd1 ; C= 48'd1; D=18'd10;
209 PCIN=48'd10;
210 OPMODE[6]=1'b0;
211 OPMODE[4]=1'b1;
212 OPMODE[1:0] =2'b01;
213 OPMODE[3:2]=2'b01;
214 OPMODE[7]=1'b0;
215 OPMODE[5]=1'b1;
216 W1=D+B;
217 W2=W1*A;
218 W3=W2+PCIN;
219 W4=W3+OPMODE[5];
220 repeat(4) begin

```



```

221     @(negedge CLK);
222 end
223 if (P!=W4)
224     begin
225         $display ("error");
226         $display("P=%d,W3=%d",P,W3);
227         $stop;
228     end
229     else begin
230 $display("success");
231 end
232 //////////////////////////////////////
233 //test second adder (add) , first adder (add) , p as input // P+PCIN=;
234 A=18'd1; B=18'd1 ; C= 48'd1; D=18'd10;
235 PCIN=48'd10;
236 OPMODE[6]=1'b0;
237 OPMODE[4]=1'b1;
238 OPMODE[1:0] =2'b10;
239 OPMODE[3:2]=2'b01;
240 OPMODE[7]=1'b0;
241 OPMODE[5]=1'b0;
242
243 repeat(4) begin
244 W3=P+PCIN;
245     @(negedge CLK);
246
247 end
248 if (P!=W3)
249     begin
250         $display ("error");
251         $display("P=%d,W3=%d",P,W3);
252         $stop;
253     end
254     else begin
255 $display("success");
256 end
257 //////////////////////////////////////
258 //test second adder (add) , first adder (add) , select is 0 in mux x and z
259 A=18'd1; B=18'd1 ; C= 48'd1; D=18'd10;
260 PCIN=48'd10;
261 OPMODE[6]=1'b0;
262 OPMODE[4]=1'b1;
263 OPMODE[1:0] =2'b00;
264 OPMODE[3:2]=2'b00;
265 OPMODE[7]=1'b0;
266 OPMODE[5]=1'b0;
267
268 repeat(4) begin
269     @(negedge CLK);
270 end
271 if (P!=0)
272     begin
273         $display ("error");
274         $display("P=%d,W3=%d",P,W3);
275         $stop;
276     end

```

```

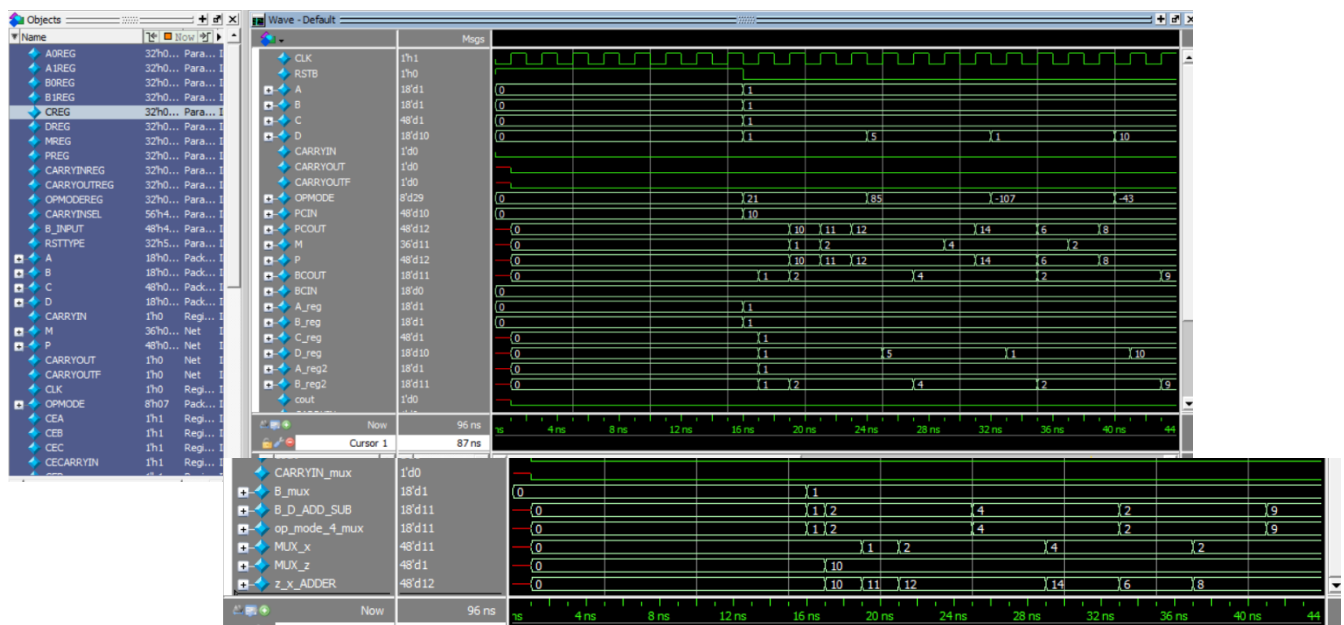
276     end
277     else begin
278     $display("success");
279     end
280     ///////////////////////////////////
281     //test second adder (add) , first adder (add) , select is 11 in mux z
282     //((D+B)*A)+C
283     A=18'd1; B=18'd1 ; C= 48'd1; D=18'd10;
284     PCIN=48'd10;
285     OPMODE[6]=1'b0;
286     OPMODE[4]=1'b1;
287     OPMODE[1:0] =2'b01;
288     OPMODE[3:2]=2'b11;
289     OPMODE[7]=1'b0;
290     OPMODE[5]=1'b0;
291     W1=D+B;
292     W2=W1*A;
293     W3=W2+C;
294     repeat(4) begin
295     @(negedge CLK);
296     end
297     if (P!=W3)
298     begin
299     $display ("error");
300     $display("P=%d,W3=%d",P,W3);
301     $stop;
302     end
303     else begin
304     $display("success");
305     end
306     ///////////////////////////////////
307     // X sel = 3 [D:A:B] && Z sel = 01[0]
308     A = 18'd1; B = 18'd1 ; C = 48'd1; D = 18'd1;
309     OPMODE[6] = 1'b0;
310     OPMODE[4] = 1'b0;
311     OPMODE[1:0] = 2'b11;
312     OPMODE[3:2] = 1'b01;
313     OPMODE[7] = 1'b0; // Add
314     OPMODE[5] = 1'b0;
315     W2={D[11:0],A[17:0],B[17:0]};
316     W3=W2+PCIN;
317     repeat(4) begin
318     @(negedge CLK);
319     end
320     if (P!=W3)
321     begin
322     $display ("error");
323     $display("P=%d,W3=%d",P,W3);
324     $stop;
325     end
326     else begin
327     $display("success");
328     end
329     $stop;
330     end
331 endmodule
332

```

3. Do file:

```
vlib work
vlog project_1.v reg+mux.v DSP_tb.v
vsim -voptargs=+acc work.DSP_tb
add wave *
run -all
#quit -sim
```

4. Waveform:



5. Transcript:

```
VSIM 44> run -all
# success
# success
# success
# success
# success
# success
# success
# success
# success
```

Vivado and synthesis part:

1. Constrains file:

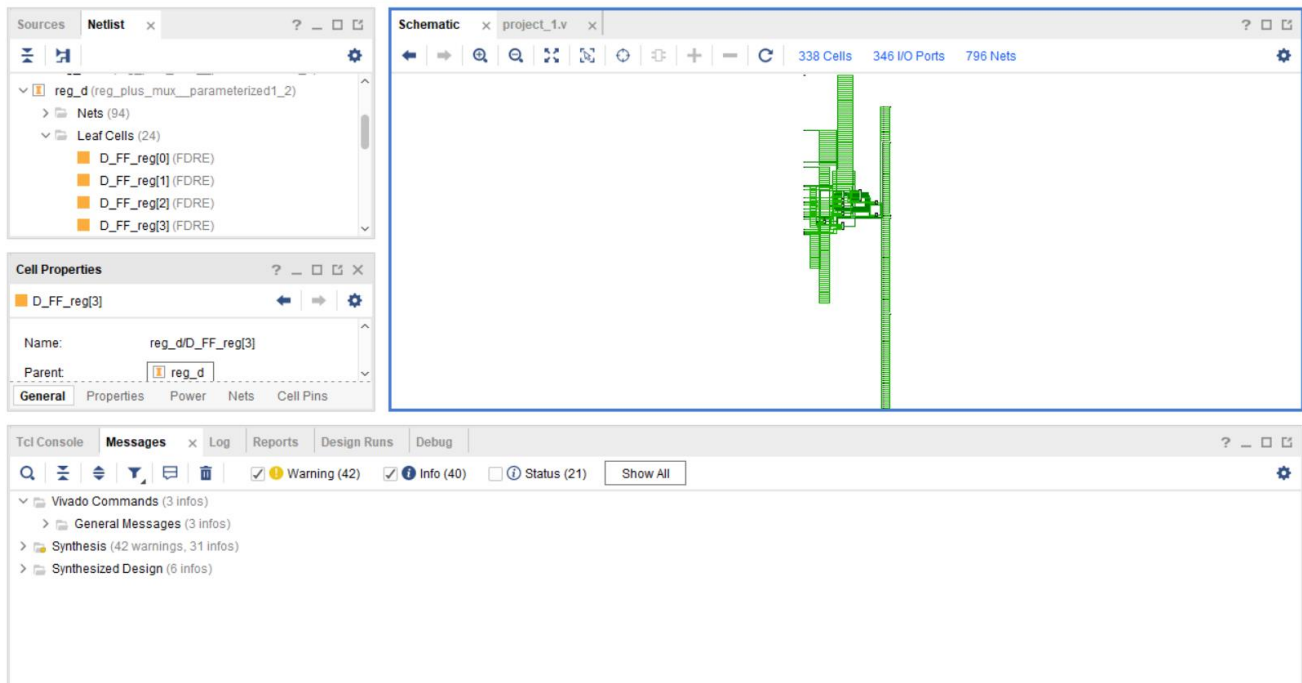
```
4 ## - rename the used ports (in each line, after get_ports) according to the top level signal names in
5
6 ## Clock signal
7 set_property -dict { PACKAGE_PIN W5 IOSTANDARD LVCMOS33 } [get_ports CLK]
8 create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports CLK]
9
10
11 ## Switches
12 #set_property -dict { PACKAGE_PIN V17 IOSTANDARD LVCMOS33 } [get_ports {in0[0]}]
13 #set_property -dict { PACKAGE_PIN V16 IOSTANDARD LVCMOS33 } [get_ports {in0[1]}]
14 #set_property -dict { PACKAGE_PIN W16 IOSTANDARD LVCMOS33 } [get_ports {in1[0]}]
15 #set_property -dict { PACKAGE_PIN W17 IOSTANDARD LVCMOS33 } [get_ports {in1[1]}]
```

2. Elaboration:

The screenshot displays the Vivado IDE interface during the elaboration phase. The top-left pane shows the 'Netlist' view with a hierarchical tree of components: DSP, Nets (857), Leaf Cells (14), and several parameterized multiplexers (reg_plus_mux). The top-right pane shows the 'Schematic' view, which is a complex logic diagram with numerous logic blocks, multiplexers, and interconnecting green lines representing signals. The bottom pane shows the 'Messages' window, which contains a list of status messages and warnings. The messages include information about Vivado commands, elaborated design warnings, synthesis warnings, and implementation details like 'Design Initialization' and 'Loading part xc7a200tfg1156-3'.

3. Synthesis:

- Schematic:



- Report utilization:

Name	Slice LUTs (134600)	Slice Registers (269200)	DSP s (740)	Bonded IOB (500)	BUFGCTRL (32)
▼ DSP	255	160	1	327	1
reg_a1 (reg_plus_mux...	0	18	0	0	0
reg_b1 (reg_plus_mux...	0	18	0	0	0
reg_c (reg_plus_mux...	0	48	0	0	0

- Timing report:

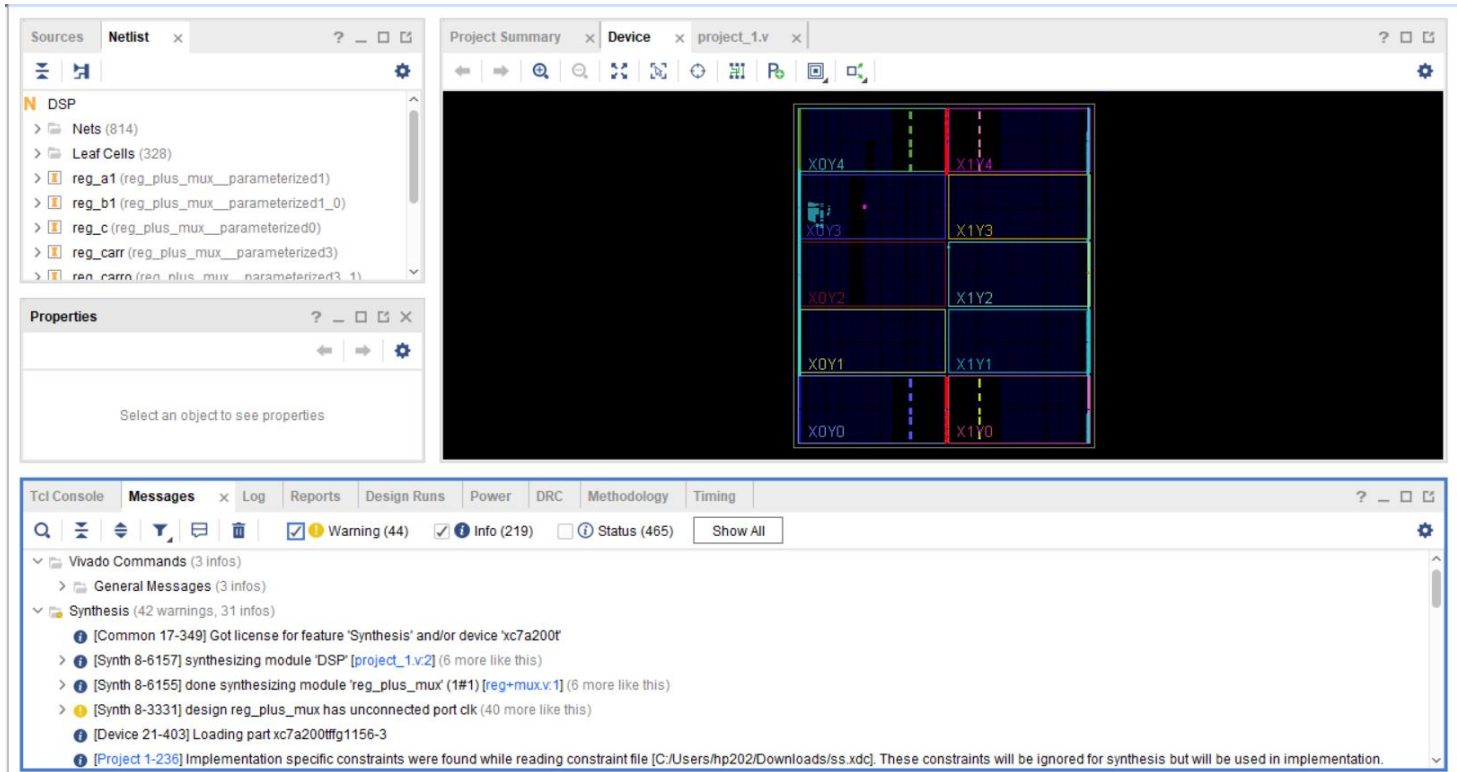
Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.216 ns	Worst Hold Slack (WHS): 0.182 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 106	Total Number of Endpoints: 106	Total Number of Endpoints: 162

All user specified timing constraints are met.

4. Implementation:

- Schematic:



- Timing:

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 3.713 ns	Worst Hold Slack (WHS): 0.273 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 125	Total Number of Endpoints: 125	Total Number of Endpoints: 181

All user specified timing constraints are met.

- Utilization:

Name	Slice LUTs (133800)	Slice Registers (267600)	Slice (33450)	LUT as Logic (133800)	LUT Flip Flop Pairs (133800)	DSP s (740)	Bonded IOB (500)	BUFGCTRL (32)
DSP	254	179	106	254	26	1	327	1
reg_a1 (reg_plus_mux...)	0	18	6	0	0	0	0	0
reg_b1 (reg_plus_mux...)	0	36	12	0	0	0	0	0
reg_c (reg_plus_mux...)	0	48	14	0	0	0	0	0

